

Docker Compose

නියමයි! අපි දැන් Docker වල තියෙන ඊළඟ ලොකුම කඩඉම වෙන **docker-compose** ගැන කතා කරමු.

මෙවිට වෙලා අපි ඉගෙන ගත්තේ එක Container එකක් (උදාහරණයක් විදිහට Angular app එක විතරක්) ලෑප් එකේ run කරන විදියෙනෙ. හැබැයි ඇත්ත ලෝකයේදී අපි හදන Software Projects ඊට වඩා ගොඩක් සංකීර්ණයි.

1. මොකක්ද මේ ප්රශ්නය? 🧑🏻♂️

උදාහරණයක් විදිහට Microservices පාවිච්චි කරලා හදන Hotel Management System එකක් හරි, Full-stack POS system එකක් හරි ගත්තොත්, ඒකෙ එක Container එකක් නෙමෙයි තියෙන්නේ:

- **Frontend එකට** එක Container එකක් (React / Next.js / Angular).
- **Backend එකට** තව Container එකක් (Spring Boot / Node.js).
- **Database එකට** තව එකක් (MongoDB / MySQL).
- සමහරවිට **Authentication** වලට (Keycloak වගේ) තව එකක්.

මේ ඔක්කොම වෙන වෙනම docker build කර කර, port ගණන් හදාගෙන docker run කර කර ඉන්න ගියොත් ඒක ලොකු වදයක්. ඒ වගේම මේ Containers වලට එකිනෙකා එක්ක කතා කරන්න (Network කරන්න) ඕනෙත් (උදාහරණයක් විදිහට Backend එකට Database එකත් එක්ක කතා කරන්න ඕනේ). මේවා Terminal එකෙන් අතින් හදන එක ගොඩක් අමාරුයි.

2. විසඳුම: docker-compose 🚀

අන්න ඒ ප්රශ්නට විසඳුම තමයි Docker Compose. මේකෙන් කරන්නේ අර Containers ඔක්කොම (අපි මේවට **"Services"** කියලා කියනවා) එකම එක file එකකින් පාලනය කරන එකයි. ඒ file එකේ නම තමයි **docker-compose.yml**.

මේක ලියන්නේ YAML format එකෙන්. මේකේ { } වරහන් නැහැ, ඒ වෙනුවට තියෙන්නේ හිස්තැන් (spaces / indentation) විතරයි.

අපි සම්පූර්ණ Full-stack App එකකට (Frontend + Backend + Database) අදාළ docker-compose.yml file එකක් ලියවෙන විදිය බලමු:

```

1 version: '3.8' # Docker compose version එක
2
3 services:
4   # 1. Database Service (උදා: MongoDB)
5   database:
6     image: mongo:latest      # මේකට වෙනම Dockerfile එකක් නෑ, කෙලින්ම official image එක ගන්නවා
7     ports:
8       - "27017:27017"
9
10  # 2. Backend Service (උදා: Node.js / Spring Boot)
11  backend:
12    build: ./backend-folder    # Backend එකේ අපි ලියපු Dockerfile එක තියෙන folder එක
13    ports:
14      - "5000:5000"
15    depends_on:
16      - database              # Database එක Start වුණාට පස්සෙයි Backend එක Start වෙන්නේ
17
18  # 3. Frontend Service (උදා: React / Next.js)
19  frontend:
20    build: ./frontend-folder   # Frontend එකේ Dockerfile එක තියෙන folder එක
21    ports:
22      - "3000:3000"           # අපේ බ්‍රවුසරෙන් යන්නේ http://localhost:3000 ට
23    depends_on:
24      - backend               # Backend එක Start වුණාට පස්සෙයි Frontend එක Start වෙන්නේ

```

3. මේකෙ තියෙන "මැජික්" එක මොකක්ද? ✨

මේ docker-compose.yml file එක ඔයාගේ project එකේ ජ්රධාන (Root) folder එකේ හැදුවට පස්සේ, ඔයාට අර කලින් වගේ දිග commands ගහ ගහ ඉන්න ඕනේ නැහැ. Terminal එකේ මේ command එක ගැහුවම ඇති:

```

Bash

# මේ services ඔක්කොම එකපාර හදලා run කරන්න:
docker-compose up -d

```

මේක ගැහුවම Docker එකෙන් තනියම:

1. Backend සහ Frontend වලට අදාළ Images දෙක අර Dockerfile වලට ගිහින් build කරගන්නවා.
2. Official MongoDB image එක download කරගන්නවා.
3. මේ තුනම දාලා Containers 3 ක් start කරනවා.
4. **වැදගත්ම දේ:** මේ තුනම එකිනෙකට කතා කරන්න පුළුවන් විදිහට එකම **Internal Network** එකකට connect කරනවා!

ඔයාට වැඩ ඉවර වෙලා මේ මුළු system එකම එකපාර නවත්තලා දන්න ඕනෙනම්:

```
Bash

docker-compose down
```

ගොඩක් ලේසියි නේද? වෙන වෙනම run කරනවාට වඩා මේකෙන් අපේ කාලය සැහෙන් ඉතුරු වෙනවා.

Docker Network එක ඇතුළේ **Backend** එක කොහොමද හරියටම **Database** එකට **connect** වෙන්නේ? මට මේක නිකන් **example** එකක් විදිහට **Spring Boot project** 4ක් තියෙන **folder** එකකම පැහැදිලි කරලා, ඒවා කොහොමද **connect** වෙලා වැඩ කරන්නේ කියන එක පැහැදිලි කරන්න. මුලින් මේක කතා කරලා පස්සේ **Volume** වලට යමු.

English: How exactly does the **Backend connect** to the **Database** inside a **Docker Network**? Please explain this using an **example** with 4 **Spring Boot projects** in the same **folder**, and explain how they **connect** and work together. Let's discuss this first and move to **Volumes** later.

නියමයි! Spring Boot microservices 4ක් තියෙන project එකක් කියන්නේ මේක තේරුම් ගන්න පුළුවන් හොඳම සහ ප්රායෝගිකම උදාහරණයක්.අපි හිතමු ඔයා හදන්නේ Microservices architecture එකක් පාවිච්චි කරන ලොකු **Hotel Management System** එකක් කියලා.

මේකෙදි අපිට Spring Boot projects 4කුයි, ඒකට අදාළ Database එකකුයි තියෙනවා. අපි මේක පියවරෙන් පියවර ලස්සනට තේරුම් ගමු.

1. Folder Structure එක (අපේ Project එකේ හැඩය) 📁

ඔයාගේ ප්රධාන folder එක (Root directory) ඇතුළේ තියෙන විදිය මුලින්ම බලමු:

```
hotel-management-system/
|
|_____ docker-compose.yml    <-- මුළු system එකම පාලනය කරන file එක
|
|_____ eureka-server/        <-- Spring Boot Project 1 (Service Registry)
|   |_____ Dockerfile
```

```

|   └── src/
|
|
└── api-gateway/      <-- Spring Boot Project 2 (Gateway)
    ├── Dockerfile
    └── src/
    |
    ├── guest-service/      <-- Spring Boot Project 3 (Guests ලබා manage කරන එක)
    │   ├── Dockerfile
    │   └── src/
    │   |
    │   └── booking-service/      <-- Spring Boot Project 4 (Bookings manage කරන එක)
    │       ├── Dockerfile
    │       └── src/
    |
    └──

```

2. Docker Network එකේ "රහස්" (The Golden Rule) 🤔

සාමාන්යයෙන් ඔයාගේ ලෑප් එකේ Spring Boot run කරද්දී ඔයා Database එකට connect වෙන්නේ localhost:3306 කියලානේ.

හැබැයි Docker ඇතුලට ගියාම **localhost කියන එකේ තේරුම වෙනස් වෙනවා!** Guest-service container එක ඇතුලේ ඉඳන් localhost කිව්වොත්, ඒකෙන් අදහස් වෙන්නේ "Guest-service container එක ඇතුලෙම" කියන එකයි. එතන Database එකක් නැහැනේ.

එහෙනම් කොහොමද Connect වෙන්නේ? Docker Compose වලින් අර containers ඔක්කොම එකම **Internal Network** (අතප්පය ජාලයක්) එකකට දානවා. ඊටපස්සේ ඒ network එක ඇතුලේ කතා කරන්න පාවිච්චි කරන්නේ අර docker-compose.yml එකේ අපි දෙන **Service Name එකමයි!** (මේකට කියන්නේ Docker Internal DNS කියලා).

=====

Sinhala: Guest-service container එක කියන්නේ මොකක්ද? එහෙම එකක් කොහේද තියෙන්නේ අර folder structure එකේ?

English: What is the **Guest-service container**, and where is such a thing located in that **folder structure**?

ඔයා අහපු ප්රශ්නය හරිම සාධාරණයි! ගොඩක් අයට මුලින්ම මේක පැටලෙනවා.

කෙටි උත්තරය තමයි: "**Guest-service container**" එක අර **Folder Structure** එක ඇතුලේ **කොහේවත් නැහැ!** 🤔

මේක තේරුම් ගන්න අපි Folder එකයි, Container එකයි අතර තියෙන වෙනස බලමු:

1. Folder එක කියන්නේ මොකක්ද? 📁

අර අපි දැක්ක guest-service/ කියන්නේ ඔයාගේ ලැප් එකේ Hard Disk එකේ තියෙන සාමාන්ය folder එකක් විතරයි. එක ඇතුලේ තියෙන්නේ ඔයා ලියපු Java code එක, pom.xml එක සහ Dockerfile එක.

- **සරල උදාහරණයක්:** මේක හරියට ගෙයක් හදන්න කොළයක ඇඳපු **ප්ලෑන් එක (Blueprint)** වගේ. ප්ලෑන් එක ඇතුලේ මිනිස්සුන්ට ජීවත් වෙන්න බැහැනේ, එක නිකමිම සටහනක් විතරයි.

2. Container එක කියන්නේ මොකක්ද? 📦

Container එකක් කියන්නේ අර Folder එකේ තියෙන Dockerfile එක කියවලා, ඒකෙ තියෙන විදියට ඔක්කොම ලැස්ති කරලා, **ඇත්තටම Run වෙන (පණ තියෙන) Process එකටයි.** මේක තියෙන්නේ ඔයාගේ ලැප් එකේ RAM එකෙයි, Docker Engine එක ඇතුලෙයි.

- **සරල උදාහරණයක්:** ප්ලෑන් එක බලලා, ගඩොල් සිමෙන්ති දාලා හදපු **ඇත්තම ගෙදර** තමයි Container එක කියන්නේ.

එහෙනම් අර docker-compose.yml එකේ අපි මොකක්ද ලිව්වේ?



අපි docker-compose.yml file එකේ මෙන්න මෙහෙම ලිව්වා මතකද:

```
1 guest-service: # <-- මේක තමයි හැදෙන Container එකට අපි දෙන නම
2 | build: ./guest-service # <-- මේක තමයි අපේ Hard disk එකේ තියෙන Folder එකට යන තැන
```

මෙතනදී docker-compose up ගැනුවම Docker එකෙන් කරන්නේ මේ ටිකයි:

1. "ආ මට ./guest-service කියන **Folder එකට** යන්න කියලා තියෙනවා."
2. "මම ඒ Folder එක ඇතුලේ තියෙන Dockerfile එක (ප්ලෑන් එක) කියවලා ඒ විදිහට Image එකක් හදාගන්නවා."
3. "ඊටපස්සේ මම ඒ Image එකෙන් අලුත් **Container එකක් (ඇත්ත ගේ) Start කරනවා.**"
4. "අන්නිමට ඒ Start කරපු අලුත් Container එකට මම guest-service කියලා නමකුත් දානවා."

ඒ කියන්නේ අර Folder එකෙන් තමයි Container එක ඉපදෙන්නේ. හැබැයි Container එක කියන්නේ අර Folder එක නෙමෙයි!

3. docker-compose.yml එක ලියවෙන විදිය

අපි මේ projects 4 යි, MySQL Database එකයි run කරන්න ලියන file එක බලමු:

```
# 1. Eureka Server එක
eureka-server:
  build: ./eureka-server    # eureka folder එකේ තියෙන Dockerfile එකෙන් build කරන්න
  ports:
    - "8761:8761"

# 2. API Gateway එක
api-gateway:
  build: ./api-gateway
  ports:
    - "8080:8080"
  depends_on:
    - eureka-server          # Eureka start වුණාට පස්සේ මේක start වෙන්න

# 3. Guest Service එක (මේක Database එකට connect වෙනවා)
guest-service:
  build: ./guest-service
  ports:
    - "8081:8081"
  depends_on:
    - hotel-db                # DB එක start වුණාට පස්සේ මේක start වෙන්න
    - eureka-server

# 4. Booking Service එක (මේකත් Database එකට connect වෙනවා)
booking-service:
  build: ./booking-service
  ports:
    - "8082:8082"
  depends_on:
    - hotel-db
    - eureka-server
```

4. Spring Boot එකට Database එකේ පාර පෙන්නන්නේ කොහොමද?

දැන් තමයි වැදගත්ම තැන. ඔයාගේ guest-service එකේ තියෙන application.properties (නැත්නම් application.yml) file එකේ database URL එක වෙනස් වෙන්න ඕනේ මෙන්න මේ විදිහට:

කලින් (Local PC එකේදි තිබ්බ විදිය):

spring.datasource.url=jdbc:mysql://localhost:3306/hotel_db

Docker එකට දැමීමට පස්සේ (අලුත් විදිය): spring.datasource.url=jdbc:mysql://hotel-db:3306/hotel_db

ඔයා දැක්කද වෙනස? මම localhost වෙනුවට **hotel-db** කියලා දුන්නා. ඒ නම ආවේ අර docker-compose.yml එකේ මම Database service එකට දීපු නමෙනයි!

මේක වැඩ කරන්නේ කොහොමද? (පසුබිම් කතාව) ⚙

1. ඔයා docker-compose up -d ගැනුවම, Docker එකෙන් තනියම "hotel-network" කියලා virtual WiFi network එකක් වගේ එකක් හදනවා.
2. ඊටපස්සේ අර containers 5 ම ඒ network එකට connect කරනවා.
3. Guest Service එක Spring Boot හරහා hotel-db ට කතා කරන්න හැදුවම, Docker එකේ ඇතුලේ තියෙන DNS Server එකෙන් තේරුම් ගන්නවා "ආ මේ හොයන්නේ අපේ අතින් Container එක වෙත MySQL එක නේද" කියලා.
4. ඊටපස්සේ ඒක හරියටම අදාළ තැනට traffic එක යවනවා.

මේ විදිහට eureka-server එකට connect වෙන්න ඕන වුණොත්, localhost:8761 වෙනුවට http://eureka-server:8761/eureka කියලා දුන්නම හරියටම වැඩ කරනවා.

=====

Sinhala: මට පොඩි ගැටලුවක් තියෙනවා

spring.datasource.url=jdbc:mysql://localhost:3306/hotel_db ඇයි hotel_db වුණේ? කවුද මේ නම මේකට දුන්නේ? ඒ නම ඒකට එන්නේ කොහොමද?

English: I have a small problem: why did

spring.datasource.url=jdbc:mysql://localhost:3306/hotel_db become hotel_db? Who gave this name to it? How does that name get assigned to it?

=====

නියම ප්රශ්නයක්! ඔයා හරිම තියුණු විදිහට දේවල් අල්ලගන්නවා. 🤖

මෙතනදි hotel_db කියන්නේ MySQL Server එක ඇතුලේ තියෙන **Database එකේ (Schema එකේ) නමයි.**

මේ නම මේකට දුන්නේ **අපිමයි!** ඔයාට මතකද අපි කලින් ලියපු docker-compose.yml file එකේ Database එකට අදාළව ලියපු කොටස?

අපි ඒක ආයෙත් බලමු:

```

1 hotel-db: # <-- මේක Container එකේ (Server එකේ) නම (Hyphen එකක් තියෙනවා)
2 image: mysql:8.0
3 environment:
4   MYSQL_ROOT_PASSWORD: rootpassword
5   MYSQL_DATABASE: hotel_db # <-- මෙන්න මෙතන තමයි අපි Database එකේ නම දුන්නේ! (Underscore එකක් තියෙනවා)|

```

මේක ඇත්තටම වෙන්වේ කොහොමද? ⚙️

1. ඔයා docker-compose up ගහලා මුලින්ම මේ MySQL container එක start කරද්දී, Docker එකෙන් අර environment යටතේ දීලා තියෙන variables ටික කියවනවා.
2. එතන MYSQL_DATABASE: hotel_db කියලා තියෙනවා දැක්කම, MySQL Server එක start වෙන ගමන්ම තනියම **හිස් Database එකක් අලුතින්ම හදනවා** (හරියට CREATE DATABASE hotel_db; කියන SQL command එක යටින් run වෙනවා වගේ).
3. ඊටපස්සේ ඔයාගේ Spring Boot app එකේ application.properties එකේ අපි අර නම දෙනවා: spring.datasource.url=jdbc:mysql://hotel-db:3306/hotel_db

අපි මේ Connection URL එක කැලි වලට කඩලා බලමු: 🌿

- **jdbc:mysql://** -> කතා කරන භාෂාව (Protocol එක).
- **hotel-db** -> කතා කරන්න ඕන Server එක (අපේ MySQL Container එකේ නම).
- **:3306** -> Server එකට ඇතුල් වෙන දොර (Port එක).
- **/hotel_db** -> අර Server එක ඇතුලේ තියෙන **නිශ්චිත Database එක!** (අර කලින් හැදුණු හිස් Database එක).
-

කෙටියෙන්ම කිව්වොත්: මේ නම් දෙක අනිවාර්යයෙන්ම සමාන වෙන්න ඕනේ. ඔයා docker-compose.yml එකේ MYSQL_DATABASE: my_super_hotel කියලා දුන්නොත්, ඔයා Spring Boot එකේ URL එක අගටත් ...:3306/my_super_hotel කියලා අනිවාර්යයෙන්ම වෙනස් කරන්න ඕනේ. නැත්නම් "Unknown database" කියලා error එකක් එනවා.\


```
# 1. Eureka Server එක
eureka-server:
  build: ./eureka-server      # eureka folder එකේ තියෙන Dockerfile එකෙන් build කරන්න
  ports:
    - "8761:8761"

# 2. API Gateway එක
api-gateway:
  build: ./api-gateway
  ports:
    - "8080:8080"
  depends_on:
    - eureka-server           # Eureka start වුණාට පස්සේ මේක start වෙන්න

# 3. Guest Service එක (මේක Database එකට connect වෙනවා)
guest-service:
  build: ./guest-service
  ports:
    - "8081:8081"
  depends_on:
    - hotel-db                 # DB එක start වුණාට පස්සේ මේක start වෙන්න
    - eureka-server

# 4. Booking Service එක (මේකත් Database එකට connect වෙනවා)
booking-service:
  build: ./booking-service
  ports:
    - "8082:8082"
  depends_on:
    - hotel-db
    - eureka-server
```

දැන් මේ **code** එකෙන් **container** 4ක් (ඇත්තටම මේකේ **Database** එකත් එක්ක 5ක් තියෙනවා) වෙන වෙනම හැඳෙනවාද, නැත්නම් ඔක්කොම එකම තනි **container** එකක් විදිහට හැඳෙනවාද?

English: Now, from this **code**, are 4 **containers** (actually 5 including the **Database**) created separately, or are they all created together as a single **container**?

මේකෙන් හැඳෙන්නේ **තනි Container එකක් නෙමෙයි!** මේකෙන් **වෙන වෙනම Containers 5 ක්** හැදිලා, ඒ ඔක්කොම එකම Network එකකින් එකිනෙකාට සම්බන්ධ වෙනවා (Connect වෙනවා).

Docker වල තියෙන රන්වන් නීතියක් (Golden Rule) තමයි: "**එක Container එකක තියෙන්න ඕනේ එක ජර්මාන Process එකක් (වැඩක්) විතරයි**" කියන එක. ඒ නිසා අපි කවදාවත් Database එකයි, Spring Boot apps ටිකයි ඔක්කොම එකම පෙට්ටියකට දාන්නේ නැහැ.

ඔයාගේ Code එකට අනුව හරියටම වෙන්නේ මේකයි:

1. **hotel-db Container එක:** MySQL විතරක් run වෙන වෙනමම Container එකක් හැඳෙනවා.

2. **eureka-server Container එක:** Eureka server එක විතරක් run වෙන තව Container එකක් හැඳෙනවා.
3. **api-gateway Container එක:** මේකට තව Container එකක් හැඳෙනවා.
4. **guest-service Container එක:** මේකට තව Container එකක් හැඳෙනවා.
5. **booking-service Container එක:** මේකටත් වෙනම Container එකක් හැඳෙනවා.

එහෙනම් මේවා එකට එකතු වෙන්නේ කොහොමද? 🤝

අපි docker-compose up කියන command එක ගැහුවම, Docker එකෙන් යටින් මේ දේවල් ටික සිද්ධ කරනවා:

1. මුලින්ම එයා තනියම **අලුත් Virtual Network එකක්** හඳුනවා (මේක හරියට ඔයාගේ ලෑප් එක ඇතුළේ අලුතින් අදාශයමාන WiFi router එකක් දැමීමා වගේ වැඩක්).
2. ඊටපස්සේ අර අපි කතා කරපු Containers 5 ම හදලා, ඒ 5 ම අර අලුත් Network එකට Connect කරනවා.
3. දැන් මේ Containers 5 ම ඉන්නේ එකම ගෙදරක (Network එකක) කාමර 5 ක වගේ. ඒ නිසා එයාලට අර අපි කලින් කතා කරපු විදිහට hotel-db හෝ eureka-server වගේ නම් (Service names) භාවිත කරලා ලේසියෙන්ම එකිනෙකාට කතා කරන්න පුළුවන්.

ඇයි අපි ඔක්කොම එක Container එකකට දාන්නේ නැත්තේ? 🤔

හිතන්න, guest-service එකට එකපාරටම ලොකු traffic එකක් ඇවිත් ඒක crash වුණා කියලා. ඔයා ඔක්කොම එකම Container එකක දාලා තිබ්බා නම්, මුළු Hotel Management System එකම, Database එකත් එක්කම down වෙනවා.

හැබැයි මේ විදිහට වෙන වෙනම Containers වල තිබ්බම, guest-service Container එක විතරක් down වෙනවා, අනිත් Containers 4 කිසිම ප්රශ්නයක් නැතුව දිගටම run වෙනවා. ඒ වගේම අපිට ඕනෙනම් guest-service Containers 3 ක් 4 ක් එකපාර run කරලා load එක බෙදලා යවන්නත් (Load balancing) මේ ක්රමය නිසා පුළුවන්.

=====

Sinhala: docker-compose up කියලා ගැහුවාම **output** එක විදිහට **Container 5ක්** වෙන වෙනම පෙන්වනවාද? නැත්නම් එක **Container** එකක් ඇතුළේ **Container 5ක්** දාලා ඉන්නවාද? මොකක්ද **output** එක?

English: When running docker-compose up, does it show 5 **Containers** separately in the **output**? Or are 5 **Containers** placed inside a single **Container**? What exactly is the **output**?

=====

මෙතනදි හැඳෙන්නේ **වෙන වෙනම තියෙන ස්වාධීන Containers 5 ක්!** 🙌

එක Container එකක් (ලොකු පෙට්ටියක්) ඇතුළේ තව Containers 5 ක් (පොඩි පෙට්ටි) දාන්නේ නැහැ. ඒක Docker වල ප්රතිපත්තියට සම්පූර්ණයෙන්ම විරුද්ධයි.

ඔයා Terminal එකේ docker-compose up කියලා ගැහුවම, ඔයාගේ ඇස් පනාපිටම මේ Containers 5 වෙන වෙනම හැඳෙන විදිය Output එකේ ලස්සනට පෙන්නනවා.

ඒ Output එක දළ වශයෙන් මේ වගේ තමයි පේන්නේ:

Creating network "hotel-project_default" with the default driver

Creating hotel-db ... done

Creating eureka-server ... done

Creating api-gateway ... done

Creating guest-service ... done

Creating booking-service ... done

Attaching to hotel-db, eureka-server, api-gateway, guest-service, booking-service

hotel-db | 2026-03-08T09:30:00.000000Z 0 [Note] mysqld: ready for connections.

eureka-server | 2026-03-08 15:00:05.123 INFO 1 --- [main] c.n.e.EurekaServerApplication :
Started EurekaServerApplication...

guest-service | 2026-03-08 15:00:10.456 INFO 1 --- [main] c.h.g.GuestServiceApplication :
Started GuestServiceApplication...

...

ඔයාට පෙනවා නේද මුලින්ම ඒ ගොල්ලන්ට කතා කරන්න Network එකක් හදාගෙන, ඊටපස්සේ එකින් එක Containers 5 වෙන වෙනම හදනවා. අන්නිමට ඒ 5 න්ම එන Logs (විස්තර) එකම Terminal එකේ කවලමේ පෙනන්නවා.

මේක හරියටම බලාගන්නේ කොහොමද? 🤔

අර Logs ඔක්කොම එකපාර පේද්දී පැටලෙන නිසා, අපි ගොඩක් වෙලාවට කරන්නේ මේක Background එකේ Run කරන එකයි.

1. ඔයා මුලින්ම ගහන්න: `docker-compose up -d`
2. ටිකකින් ඒක හැදිලා ඉවර වුණාම, අර අපි කලින් ඉගෙන ගත්ත command එක ගහලා බලන්න: **`docker ps`**

අන්න එතකොට ඔයාට ලස්සන Table එකක් (වගුවක්) එනවා. ඒකේ **පේළි 5 කින්**, එකිනෙකට වෙනස් Container IDs 5 ක් එක්ක, අර අපි හදපු Containers 5 වෙන වෙනම Run වෙනවා ඔයාට පැහැදිලිවම බලාගන්න පුළුවන්.

=====

Sinhala: මේ හැදෙන **container** 5 එකම **network** එකක් එක්ක සම්බන්ධ වෙන්නේ කොහොමද? එයාලා දැනගන්නේ කොහොමද එයාලා එක **connection** එකක ඉන්නේ කියලා? අනික මේ 5ම වෙන

වෙනම **run** කරන්න ඕනද?

English: How do these 5 created **containers** connect to the same **network**? How do they know they are in one **connection**? Also, do all 5 of these need to be **run** separately?

මෙන්න මේක තමයි Docker Compose වල තියෙන ලොකුම වාසිය! ඔයා අහපු ප්රශ්න වලට මම එකින් එක පැහැදිලි උත්තර දෙන්නම්.

1. මේ 5 ම වෙන වෙනම Run කරන්න ඕනෙද? 🏃♂️

නැහැ, කොහෙත්ම නැහැ! මේ Containers 5 ම වෙන වෙනම හදාගන්න docker run commands 5 ක් ගහන්න ඕනේ නැහැ. ඒක තමයි docker-compose.yml file එකක් පාවිච්චි කරන එකේ ප්රධානම අරමුණ.

ඔයාට තියෙන්නේ ඔයාගේ Project folder එක ඇතුළේ Terminal එක open කරලා මෙන්න මේ **එකම එක Command එක** ගහන්න විතරයි:

```
Bash

# මේ services ඔක්කොම එකපාර හදලා run කරන්න:
docker-compose up -d
```

මේ එක Command එක ගැහුවම Docker එකෙන් අර file එක කියවලා, තනියම Network එක හදලා, Containers 5 ම පිළිවෙලට (depends_on වල දීලා තියෙන විදියට) Start කරනවා. ඔක්කොම වැඩ ඉවර වුණාම මේ 5 ම එකපාර නවත්තලා මකලා දාන්න ඕනෙනම් docker-compose down කියලා ගැහුවම ඇති.

2. මේ 5 එකම Network එකකට සම්බන්ධ වෙන්නේ කොහොමද? 🌐

ඔයා අර docker-compose up command එක ගහනකොටම, Docker එකෙන් යටින් කරන පළවෙනිම වැඩේ තමයි **"Default Network"** එකක් අලුතින්ම නිර්මාණය කරන එක.

උදාහරණයක් විදිහට ඔයාගේ මුළු project එකම තියෙන folder එකේ නම hotel-system නම්, Docker එකෙන් hotel-system_default කියලා අලුත් අතප්පය ජාලයක් (Virtual Network එකක්) හදනවා. ඊටපස්සේ එයා අර අලුතින් හදන Containers 5 ම, ඒ Network එකට plug කරනවා (හරියට එකම WiFi රවුටර්කට ලැප්ටොප් 5 ක් connect කළා වගේ).

3. එයාලා එකම Network එකේ ඉන්නේ කියලා දන්නේ කොහොමද? 🧠

මේක තමයි නියම මැජික් එක! මේකට අපි කියන්නේ **Docker Internal DNS** (Domain Name System) කියලා.

හරියට ඔයාගේ ෆෝන් එකේ Contacts තියෙනවා වගේ වැඩක් මේක.

- ඔයා යාළුවෙකුට කෝල් කරද්දී එයාගේ ෆෝන් නම්බර් එක (IP Address එක) මතක තියාගන්නේ නැහැනේ. ඔයා Contacts වලින් "Amma" හරි "Kamal" හරි කියලා නම search කරලා කෝල් එක ගන්නවා. ෆෝන් එකෙන් තනියම නම්බර් එක හොයාගෙන කෝල් එක connect කරනවා.
- ඒ වගේම, අර hotel-system_default කියන Network එක ඇතුලේ Docker එකෙන් තනියම "Phonebook" එකක් (DNS Server එකක්) නඩත්තු කරනවා.

ඒ Phonebook එකේ තියෙන්නේ අර docker-compose.yml එකේ අපි දීපු නම් ටිකයි (Service names):

1. hotel-db -> අදාළ IP Address එක
2. eureka-server -> අදාළ IP Address එක
3. guest-service -> අදාළ IP Address එක

ඒ නිසා guest-service එක Database එකට කතා කරන්න ඕනේ වුණාම, එයා නිකමිම "හලෝ hotel-db, මට කතා කරන්න පුළුවන්ද?" කියලා අර Network එකට කෑගහලා අහනවා (Request එක යවනවා). එතකොට Docker DNS එකෙන් කියනවා "ආ, hotel-db ඉන්නේ මෙන්න මේ තැන" කියලා, හරියටම ටිරැෆික් එක Database Container එකට යවනවා.

දැන් මේ Containers ඔක්කොම එකම Command එකකින් පණගැන්විලා, එකිනෙකා එක්ක කතා කරන විදිය 100% ක් පැහැදිලියි නේද?

=====

Sinhala: එහෙම නෙවෙයි, දැන් මට දැනගන්න ඕන ඔන්න අපි දැන් ඕක **docker-compose** එක හැදුවා කියලා හිතන්න. **First time** docker-compose up මේක ගහලා, ඔන්න දැන් අපි **laptop** එක **off** කරලා ආයේ ඊළඟ දවසේ **on** කරලා, ආයේ මේක **run** කරන්න ඕන වුණාම, කොහොමද docker-compose up මේ **command** එක ගැහුවාම එයා දන්නේ මං මේ අර **container** 5ට අදාළවයි **network** එක හදන්න ඕන කියලා? මොකක්ද මේ වගේ **compose** කරපුවා අපි ළඟ කිහිපයක් තිබ්බොත්?

English: Not like that, now I want to know—let's assume we have created that **docker-compose** [file]. After running docker-compose up for the **first time**, suppose we turn **off** the **laptop** and turn it **on** the next day; when we need to **run** this again, how does it know to create the **network** specifically for those 5 **containers** when the docker-compose up **command** is executed? What if we have several such **composed** projects with us?

=====

මේක නම් ඇත්තටම එදිනෙදා වැඩ කරද්දී එන නියම ප්රායෝගික ප්රශ්නයක්! ගොඩක් අයට මුලින්ම මේක පැටලෙනවා.

ඔයා හිතන දේ හරි, අපේ ලෑප් එකේ docker-compose.yml files තියෙන Projects ගොඩක් (උදාහරණයක් විදිහට Hotel System එකක්, POS System එකක්, තව වෙන මොනවා හරි අත්හදා බලන එකක්) තියෙන්න පුළුවන්නේ.

එහෙනම් ඊළඟ දවසේ ලෑප් එක on කරලා docker-compose up ගැහුවම Docker එක හරියටම අර 5 හොයාගන්නේ කොහොමද? රහස තියෙන්නේ ඔයා ඒ Command එක ගහන "**තැන**" (**Working**

මේක සිද්ධ වෙන විදිය අපි පියවරෙන් පියවර බලමු:

1. Docker එකේ "ඇස් දෙක" තියෙන්නේ Terminal එකේ! 🙄

ඔයා Terminal (CMD, PowerShell, හෝ VS Code Terminal) එක open කරලා docker-compose up කියලා ගැහුවම, Docker එක මුළු ලැප් එකේම තියෙන files හොයන්නේ නැහැ.

එයා කරන්නේ **ඔයාගේ Terminal එක දැනට open වෙලා තියෙන Folder එක ඇතුළේ** docker-compose.yml කියලා file එකක් තියෙනවද කියලා බලන එක විතරයි.

- **වැරදි විදිය:** ඔයා ලැප් එක on කරලා නිකමිම CMD එක open කරලා (එතකොට path එක තියෙන්නේ C:\Users\Dilshan> වගේ) docker-compose up ගැහුවොත්, "No configuration file provided" කියලා Error එකක් එනවා.
- **හරි විදිය:** ඔයා ඊළඟ දවසේ ලැප් එක on කරලා, අර Hotel Project එක තියෙන Folder එක ඇතුළට ගිහින් (උදා: D:\Projects\hotel-system>), එතන Terminal එකක් open කරලා තමයි docker-compose up -d ගහන්න ඕනේ. අන්න එතකොට එයා ඒ folder එකේ තියෙන file එක කියවලා අර හරියටම අදාළ Containers 5 ටික start කරනවා.

2. Projects පැටලෙන්නේ නැත්තේ කොහොමද? 🛡️

හිතන්න ඔයාට Projects දෙකක් තියෙනවා:

1. D:\Projects\hotel-system
2. D:\Projects\pos-system

මේ ෆෝල්ඩර් දෙක ඇතුළෙම docker-compose.yml files දෙකක් තියෙනවා.

ඔයා hotel-system ෆෝල්ඩර් එකේ ඉඳන් command එක ගැහුවම, Docker එක තනියම මේ මුළු Project එකටම නමක් (Project Name එකක්) දෙනවා. ගොඩක් වෙලාවට ඒ නම හැදෙන්නේ **Folder එකේ නමෙන්**.

එතකොට Docker එකේ යටින් ලියවෙන්නේ මෙහෙමයි:

- "මේක hotel-system project එක. මේකේ network එකේ නම hotel-system_default."
- "මේකේ Containers වල නම් hotel-system_guest-service_1, hotel-system_hotel-db_1 වගේ වෙනවා."

ඔයා අනිත් pos-system folder එකට ගිහින් command එක ගැහුවොත්, ඒක සම්පූර්ණයෙන්ම වෙනස් අලුත් network එකක් විදිහට තමයි Docker එක අඳුරගන්නේ (pos-system_default). මේ දෙක කවදාවත් එකිනෙකාට පැටලෙන්නේ නැහැ!

3. ඊළඟ දවසේ Start වෙද්දී තියෙන වාසිය ⚡

ඔයා පළවෙනි දවසේ මේක build කරලා run කළානේ. ඊළඟ දවසේ ලෑප් එක on කරලා අර ෆෝල්ඩර් එකට ගිහින් ආයෙත් docker-compose up -d ගැනුවම, Docker එක දකිනවා "ආ, මේ hotel-system එකේ Images මම ඊයේ හැදුවා, Containers ටිකත් මගේ ගාව Stop වෙලා තියෙනවා" කියලා.

ඒ නිසා එයා ආයේ අලුතින් build කර කර ඉන්නේ නැහැ. තත්පර ගණකින් අර ඊයේ හදපු Containers 5 ම ආයෙත් Start (Wake up) කරනවා.

දැන් හරියටම පැහැදිලියි නේද ලෑප් එක on කරපු ගමන් මොකද වෙන්නේ කියලා?

මේක පැහැදිලි නම්, දැන් ඔයාට ඊළඟට එන්න ඕන ප්රධානම ප්රශ්නය මේකයි: "මම ඊයේ ලෑප් එක off කරද්දී අර Database Container එකත් stop වුණානේ. එහෙනම් ඊයේ මම System එකට ඇතුල් කරපු Guests ලගේ Data ටික අද ලෑප් එක on කරද්දී Database එකේ ඉතුරු වෙලා තියෙයිද? නැත්නම් මැකිලා ගිහින් තියෙයිද?"

අන්න ඒ ප්රශ්නට උත්තරේ සහ ඒකට විසඳුම වෙන "Docker Volumes" ගැන දැන් ඉගෙන ගමුද?

Sinhala: Volume යන්න කලින් තව ප්රශ්නයක් තියෙනවා; දැන් මෙහෙම .yaml file එකෙන් හැදෙන්නේ image නෙවෙයිද container ද? දැන් ඔයා කලින් කිව්වනේ image එකක් run වෙන කොටයි ඒක container එකක් කියලා කියන්නේ කියලා. ඉතින් .yaml file එකෙන් first time docker-compose up ගැනුවාම image හැදෙන්නේ නැද්ද project 4ට අදාළව? කෙලින්ම container ද හැදෙන්නේ?

English: I have another question before moving to volumes; now, is it images or containers that are created from this .yaml file? You said earlier that it's called a container only when an image is running. So, when docker-compose up is executed for the first time from the .yaml file, aren't images created for those 4 projects? Are containers created directly?

ඔයා අහපු ප්රශ්නය 100% ක්ම හරි! ඔයා මුල ඉඳන් ඉගෙන ගත්ත Concepts ටික නියමෙටම එකට ගළපලා තියෙනවා. 🍷

ඔයා හිතපු දේ සම්පූර්ණයෙන්ම නිවැරදියි. Container එකක් හැදෙන්න කලින්, අනිවාර්යයෙන්ම ඒකට අදාළ Image එක ලෑප් එකේ තියෙන්නම ඕනේ. Image එකක් නැතුව Container එකක් කවදාවත් හදන්න බැහැ.

එහෙනම් docker-compose up ගැනුවම ඇත්තටම වෙන්නේ මොකක්ද? මේ Command එක ඇතුලේ පියවර දෙකක් හැංගිලා තියෙනවා. අපි ඒක කඩලා බලමු:

පියවර 1: Images ටික ලෑස්ති කරගැනීම (Build / Pull) 🏗️

ඔයා පළවෙනි වතාවට docker-compose up ගහනකොට, Docker එකෙන් මුලින්ම කරන්නේ Containers හදන එක නෙමෙයි, ඒකට ඕන කරන Images 5 ලෑස්ති කරගන්න එකයි.

අපේ අර .yaml file එකේ පේළි දිහා බැලුවොත් ඒක කරන විදිය තේරෙනවා:

- **hotel-db වලට:** අපි දීලා තියෙන්නේ image: mysql:8.0 කියලා. එතකොට Docker එක කරන්නේ අලුතින් image එකක් build නොකර, කෙලින්ම Docker Hub එකෙන් ඒ MySQL image එක download (pull) කරගන්නවා.
- **අර Spring Boot Projects 4 ට:** අපි දීලා තියෙන්නේ build: ./guest-service වගේ command එකක්නේ. එතකොට Docker එක කරන්නේ ඒ folder එක ඇතුලට ගිහින්න, එතන තියෙන Dockerfile එක කියවලා හරියටම docker build command එක ගැහුවා වගේ **අලුත්ම අලුත් Images 4 ක් Build කරගන්නවා.** (මේකට ටිකක් වෙලා යනවා).

පියවර 2: Containers ටික හැදීම (Run) ▶

දැන් අර පියවර 1 ඉවර වුණාම, ඔයාගේ ලෑප් එකේ අලුතින් හැදුණු (සහ download වුණු) **Images 5 ක්** තියෙනවා. අන්න ඒ Images 5 පාවිච්චි කරලා තමයි Docker එකෙන් අර අපි කලින් කතා කරපු **Containers 5 හදලා start කරන්නේ.**

සරලවම කිව්වොත්:

docker-compose up = (අවශ්‍ය docker build commands ඔක්කොම) + (අවශ්‍ය docker run commands ඔක්කොම)

මේ command එකෙන් ඔය ලොකු වැඩ දෙකම එකපාර තනියම කරන නිසා තමයි අපිට කෙලින්ම Containers ටික run වෙනවා වගේ පේන්නේ.

තව පොඩ් රහසක්: 🤔

ඔයා ඊළඟ දවසේ ලෑප් එක on කරලා ආයෙත් docker-compose up ගැහුවොත්, ඊයේ හදපු Images ටික ලෑප් එකේ දැනටමත් තියෙන නිසා, එයා ආයෙත් අලුතින් Images Build කර කර කාලය කන්නේ නැහැ. එයා කෙලින්ම පියවර 2 ට ගිහින්නත්පර ගාණකින් Containers ටික Start කරනවා. (හැබැයි ඔයා code එක වෙනස් කරලා අලුතින්ම Image එකක් build කරන්න ඕනෙනම්, docker-compose up --build කියලා ගහන්න ඕනේ).

දැන් මේ ක්රියාවලිය 100% ක් නිරවුල් නේද? Image එකයි Container එකයි අතර තියෙන සම්බන්ධය Docker Compose වලදීත් හරියටම ඒ විදිහටම වැඩ කරනවා.

=====

Sinhala: හරි, දැන් ඔයා කිව්වනේ **Spring project** එකේ .properties එකේ localhost වෙනස් කරන්න ඕන කියලා. දැන් මුල්ම වතාවට .yaml එක **run** කරන්න docker-compose up ගහන්න කලින් අපේ **project** එකේ මෙහෙම .properties file **change** කරන්න ඕනද? එහෙම **change** කළාම **container** වලින් නැතුව ආයෙමත් **normal run** කරන්න ගියාම අපි ආයෙ .properties files ආයෙ **change** කරන්න ඕනද localhost එකට?

English: Alright, you said earlier that we need to change localhost in the **Spring project** .properties [file]. Now, before running the .yaml for the first time by executing docker-compose up, do we need to change these .properties files in our **project**? Once changed like that, if we try to **run** it **normally** again without **containers**, do we have to **change** the .properties files back to localhost?

=====

ඔයා හරියටම ප්රයෝගිකව එන ලොකුම ප්රශ්නයක් ඇහුවේ! ¹⁰⁰ ඔය විදිහට හැමතිස්සෙම localhost එකයි hotel-db එකයි මාරු කර කර ඉන්න ගියොත් ඒක එපා වෙන වැඩක් නේද?

ඒ නිසා අපි ඇත්ත වැඩ කරද්දී **කවදාවත් .properties file එක අතින් වෙනස් කර කර ඉන්නේ නැහැ!** ඒ වෙනුවට අපි පාවිච්චි කරන නියම ක්රම දෙකක් තියෙනවා. මේ ක්රම වලින් ඔයාට කිසිම file එකක් වෙනස් කරන්නේ නැතුව, සාමාන්ය විදිහට ලෑප් එකෙන් run කරන්න පුළුවන්, ඒ වගේම Docker එකෙන් run කරන්න පුළුවන්.

ක්රමය 1: Environment Variables පාවිච්චි කිරීම (ගොඩක්ම ලේසි ක්රමය)

Spring Boot වල තියෙන ලොකුම වාසියක් තමයි, .properties file එකේ තියෙන ඕනෑම දෙයක්, **Environment Variables** වලින් override කරන්න (යටපත් කරලා අලුත් අගයක් දෙන්න) පුළුවන් වීම.

1. **ඔයාගේ Spring Boot Project එකේ (application.properties):** මේකෙ ඔයා සාමාන්ය විදිහට ඔයාගේ ලෑප් එකේ වැඩ කරන විදිහටම localhost කියලා තියන්න. මේක කවදාවත් වෙනස් කරන්න එපා.

Properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/hotel_db
```

2. එතකොට ඔයා Docker නැතුව නිකමිම IDE එකෙන් Run කරද්දී කිසිම අවුලක් නැතුව ලෑප් එකේ තියෙන Database එකට Connect වෙනවා.

ඔයාගේ docker-compose.yml එකේ: අපි අර Container එක හදන තැනට, **අලුත් Environment Variable එකක්** විදිහට Docker එකට ඕන කරන URL එක දෙනවා.

```
guest-service:
  build: ./guest-service
  ports:
    - "8081:8081"
  environment:
    - SPRING_DATASOURCE_URL=jdbc:mysql://hotel-db:3306/hotel_db # <-- මෙන්න මේක අලුතින් දෙනවා
  depends_on:
    - hotel-db
```

මෙතනදි වෙන්ම මොකක්ද? Docker Compose එකෙන් Container එක Run කරනකොට, Spring Boot එකට තේරෙනවා "ආ, මගේ application.properties එකේ localhost කියලා තිබ්බට, මට එළියෙන් SPRING_DATASOURCE_URL කියලා අලුත් එකක් දීලා තියෙනවා. ඒ නිසා මම අර පරණ එක අමතක කරලා මේ අලුත් hotel-db URL එක පාවිච්චි කරනවා" කියලා.

ක්රමය 2: Spring Profiles පාවිච්චි කිරීම

ඔයාට URL එක විතරක් නෙමෙයි, වෙනස් කරන්න ඕන දේවල් ගොඩක් තියෙනවා නම්, ඔයාට files දෙකක් හදන්න පුළුවන්:

1. application-dev.properties (මේකේ localhost තියෙනවා - Local PC එකට)
2. application-docker.properties (මේකේ hotel-db තියෙනවා - Docker එකට)

ඊටපස්සේ docker-compose.yml එකේදී Docker එකට කියනවා අර දෙවෙනි file එක පාවිච්චි කරන්න කියලා:

```
YAML

environment:
  - SPRING_PROFILES_ACTIVE=docker
```

කෙටියෙන්ම උත්තරේ: නැහැ, ඔයාට file එක වෙනස් කර කර ඉන්න ඕනේ නැහැ! ඔයාගේ application.properties එක Local PC එකට හරියන්න කියලා, docker-compose.yml එක හරහා Docker එකට ඕන කරන අලුත් URL එක Environment variable එකක් විදිහට පාස් කරන එක තමයි Industry Standard (හොඳම) විදිය.

=====

Sinhala: application-docker.properties (මේකේ hotel-db තියෙනවා - **Docker** එකට)

මේ **file** එක environment: - SPRING_PROFILES_ACTIVE=docker කියන **code** එකෙන් කොහොමද අඳුරගන්නේ?

English: application-docker.properties (this has hotel-db - for **Docker**)

How does this **file** get recognized by the **code** environment: -
SPRING_PROFILES_ACTIVE=docker?

=====

හරිම ලස්සන ප්රශ්නයක්! මේක ඇත්තටම Docker වල මැජික් එකක් නෙමෙයි, **Spring Boot වලම තියෙන විශේෂ ලක්ෂණයක් (Feature එකක්).**

Spring Boot හදපු අය මේක හදලා තියෙන්නේ හරිම බුද්ධිමත් විදිහටයි. අපි බලමු මේක යටින් වැඩ කරන්නේ කොහොමද කියලා.

Spring Boot වල Naming Convention (නම් තියෙන නීතිය) 📖

Spring Boot app එකක් Start වෙද්දී එයා සාමාන්යයෙන් මුලින්ම කියවන්නේ application.properties (හෝ .yml) කියන ප්රධාන file එකයි.

හැබැයි ඔයා Spring Boot එකට කිව්වොත් "මම දැන් ඉන්නේ වෙනස් පරිසරයක (Profile එකක)" කියලා, එයා තනියම අලුත් file එකක් හොයන්න පටන් ගන්නවා. ඒක හොයන්නේ මෙන්න මේ නීතියට අනුවයි:

application-{profile-name}.properties

SPRING_PROFILES_ACTIVE=docker කයනනේ මොකකද? 🌱

ඔයා ඔයාම ලෑප එකේ application.properties එකේ spring.profiles.active=docker කයලා ලව්වොත් වෙන දෙම තමය, අප මේ Docker Compose එකේ Environment variable එකක වදහට SPRING_PROFILES_ACTIVE=docker කයලා දන්නමත් වෙනනේ. (Spring Boot වල . තයේත් properties, ලොක අකරන ලයලා _ දැමමම හරයටම සමානයි).

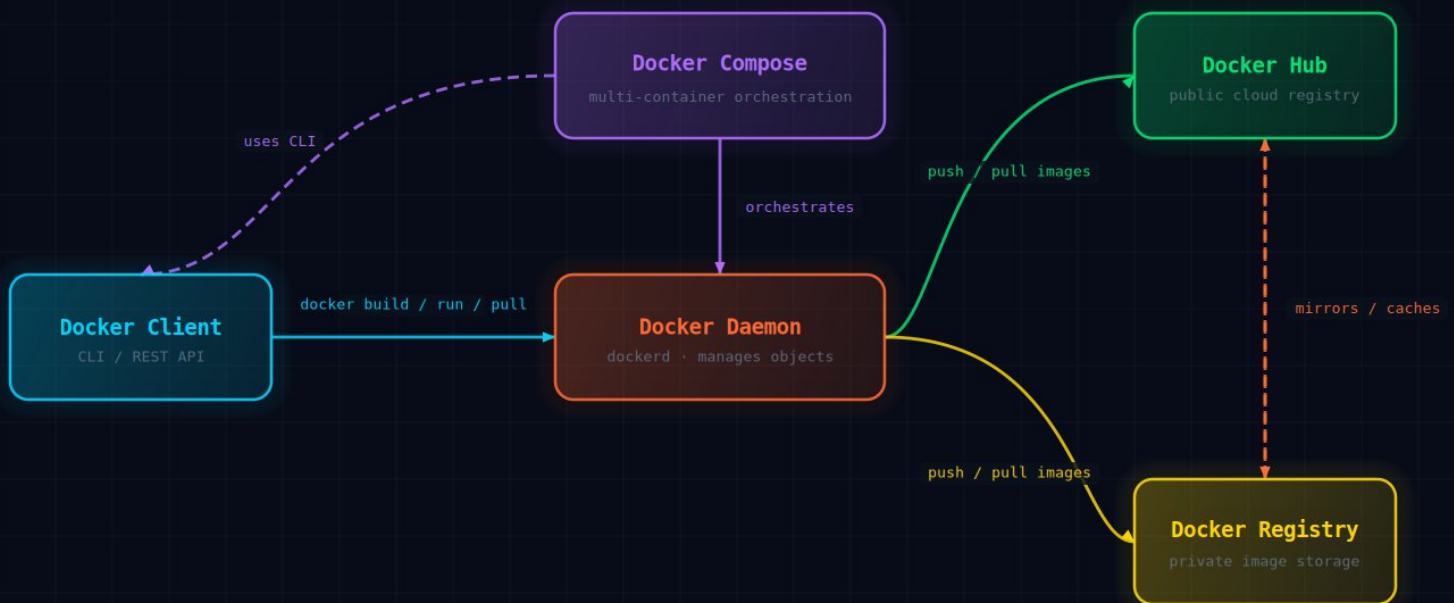
මෙක ප යවරෙන ප යවර වෙනනේ මෙහෙම ⚙️

1. **Docker Compose එකේත් Container එක Start කරනවා:**
2. එතකොට අර Container එක (පේ ටාඩ්නික් OS එක) ඇතලේ SPRING_PROFILES_ACTIVE කයන variable එකට docker කයන වචනය set වෙනවා.
3. **Spring Boot App එක Start වෙනවා:**
4. Container එක ඇතලේ Java App එක ප ණගැනවෙනකොට්ඨයා වටප ට බලලා අඳරගන්නවා "ආ.. මට docker කයන Profile එක active කරන්න කයලා නයේත්රයක ඇවත තයේනවා" කයලා.
5. **File එක හොයාගන්නවා:**
6. දැන Spring Boot එක අර මම මලන කවච නතයට අනව application- කයන කෑලලට අර Profile නම එකක කරනවා: application + - + docker + .properties.
7. ඊටප සසෙ එයා ඔයාම project එකේ ඒ නමන file එකක (ඒ කයනනේ application-docker.properties එක) තයේනවද කයලා හොයලා බලනවා.
8. **අගයන Overwrite (යටප ණකරනවා):**
9. එයා මලනම සාමානයි application.properties එක කයවනවා. ඊටප සසෙ අර හොයාගතන application-docker.properties එකක කයවනවා. දෙකෙම spring.datasource.url එක තබබොත්, එයා අර සාමානයි එක අමතක කරලා, **Docker profile එකේ තයේන අලුත URL එක (hotel-db) බාරගන්නවා!**

මේ කරමය නසා ඔයාට application-dev.properties, application-prod.properties, application-test.properties වමේ files ගොඩක තයාගෙන, ඕන කරන වෙලාවට අදාළ නම වතරක දලා ලෙසයෙනම App එකේ හැසරම වෙනස කරනන ප ළ වතකසම file එකක අතන edit කරනන ඕනේ නැහැ!

DOCKER ARCHITECTURE

Client | Daemon | Compose | Hub | Registry



Client → Daemon

Compose flow

Hub push/pull

Registry push/pull

Hub ↔ Registry